JobSheet - Week 3 - Inheritance , Abstract Class , Interface

Nama: Muhammad Fawwaz Muzakki

NIM: 251524056

Kelas: 1B

Repo GitHub: https://github.com/JackAprocat/TekProm_Praktek/tree/main/Week%203

Instruksi Pengerjaan:

1. Kerjakan 3 soal di bawah ini dengan melengkapi setiap kolom jawaban yang disediakan pada jobsheet ini.
2. Jawaban setiap soal mencakup source code, screenshot hasil dari program yang ditampilkan full screen termasuk taskbar (tambahkan beberapa screenshot jika diperlukan), penjelasan permasalahan dan solusi yang dihadapi, nama teman yang membantu memecahkan masalah (opsional).
3. Dikumpulkan pada Assignment Classroom sesuai dengan deadline yang tertera pada assignment tersebut.
4. Format penamaan file jobsheet: W3_P_<Kelas 1X>_<3 Digit_NIM_Terakhir>.docx/pdf. Contoh: W3_P_1B_001.docx/pdf.
5. Submit semua jawaban dalam bentuk file java pada repository GitHub masing-masing.

No. 1 Inheritance

Soal Praktikum
Dalam latihan ini, sebuah *subclass* bernama **Cylinder** diturunkan dari *superclass* **Circle** seperti yang ditunjukkan pada diagram kelas (di mana terdapat tanda panah yang menunjuk ke atas dari *subclass* ke *superclass*-nya). Pelajari bagaimana *subclass* Cylinder memanggil *constructor* dari *superclass* (melalui super() dan super(radius)) serta mewarisi variabel dan *method* dari *superclass* Circle. Berikut adalah kode sumber untuk **Circle.java, Cylinder.java** dan **TestCylinder.java**: **1. Circle.java** /** * The Circle class models a circle with a radius and color. */

```
public class Circle { // Save as "Circle.java"
   // private instance variable, not accessible from outside this class
   private double radius;
   private String color;

   // Constructors (overloaded)
   /** Constructs a Circle instance with default value for radius and color */
   public Circle() { // 1st (default) constructor
      radius = 1.0;
      color = "red";
   }

   /** Constructs a Circle instance with the given radius and default color */
   public Circle(double r) { // 2nd constructor
      radius = r;
      color = "red";
   }

   /** Returns the radius */
   public double getRadius() {
      return radius;
   }

   /** Returns the area of this Circle instance */
   public double getArea() {
      return radius * radius * Math.PI;
   }

   /** * Return a self-descriptive string of this instance in the form of
    * Circle[radius=?,color=?]
    */
   public String toString() {
      return "Circle[radius=" + radius + " color=" + color + "]";
   }
}
```

2. Cylinder.java

```
public class Cylinder extends Circle { // Save as "Cylinder.java"
   private double height; // private variable

   // Constructor with default color, radius and height
   public Cylinder() {
      super(); // call superclass no-arg constructor Circle()
      this.height = 1.0;
   }

   // Constructor with default radius, color but given height
   public Cylinder(double height) {
      super(); // call superclass no-arg constructor Circle()
      this.height = height;
   }

   // Constructor with default color, but given radius, height
   public Cylinder(double radius, double height) {
      super(radius); // call superclass constructor Circle(radius)
      this.height = height;
```

```
    }

    // A public method for retrieving the height
    public double getHeight() {
        return height;
    }

    // A public method for computing the volume of cylinder
    // use superclass method getArea() to get the base area
    public double getVolume() {
        return getArea() * height;
    }
}
```

3. TestCylinder.java

```
public class TestCylinder { // save as "TestCylinder.java"
    public static void main(String[] args) {

        // Declare and allocate a new instance of cylinder
        // with default color, radius, and height
        Cylinder c1 = new Cylinder();
        System.out.println("Cylinder:"
            + " radius=" + c1.getRadius()
            + " height=" + c1.getHeight()
            + " base area=" + c1.getArea()
            + " volume=" + c1.getVolume());

        // Declare and allocate a new instance of cylinder
        // specifying height, with default color and radius
        Cylinder c2 = new Cylinder(10.0);
        System.out.println("Cylinder:"
            + " radius=" + c2.getRadius()
            + " height=" + c2.getHeight()
            + " base area=" + c2.getArea()
            + " volume=" + c2.getVolume());

        // Declare and allocate a new instance of cylinder
        // specifying radius and height, with default color
        Cylinder c3 = new Cylinder(2.0, 10.0);
        System.out.println("Cylinder:"
            + " radius=" + c3.getRadius()
            + " height=" + c3.getHeight()
            + " base area=" + c3.getArea()
            + " volume=" + c3.getVolume());
    }
}
```

A. Instruksi - Modifikasi Class Circle:

1. Modifikasi class Circle, tambahkan:
    - variable color : string
    - Constructor Circle(radius : double, color : string)
    - Getter and setter untuk color
2. Implementasikan class Cylinder! Jelaskan keterhubungannya dengan class Circle!

3. Konsep apa yang diterapkan pada constructor Cylinder? Jelaskan cara kerja, kelebihan dan kekurangannya!
4. Tuliskan sebuah program pengujian (sebut saja TestCylinder) untuk menguji kelas Cylinder yang telah dibuat!

B. Instruksi – Overriding method getArea():

Metode Overriding dan "Super": Subclass **Cylinder** mewarisi metode getArea() dari superclass **Circle**.

- Cobalah melakukan *overriding* metode getArea() pada subclass Cylinder untuk menghitung luas permukaan (2π × radius × tinggi + 2 × luas alas) tabung, alih-alih luas alasnya saja!

Artinya, jika getArea() dipanggil oleh instans **Circle**, maka ia akan mengembalikan luas lingkaran. Namun, jika getArea() dipanggil oleh instans **Cylinder**, ia akan mengembalikan luas permukaan tabung.

Jika Anda melakukan *override* pada getArea() di subclass Cylinder, maka metode getVolume() tidak akan lagi berfungsi dengan benar. Hal ini terjadi karena getVolume() akan menggunakan metode getArea() yang sudah ditimpa (*overridden*) yang ditemukan di kelas yang sama. (Runtime Java hanya akan mencari ke superclass jika tidak dapat menemukan metode tersebut di kelas ini).

- Perbaikilah metode getVolume() tersebut!

Petunjuk: Setelah melakukan overriding pada metode getArea() di subclass Cylinder, Anda dapat memilih untuk memanggil metode getArea() dari superclass Circle dengan cara memanggil super.getArea().

C. Instruksi – Overriding method getArea():

Sediakan metode toString() pada kelas Cylinder, yang menimpa (override) metode toString() yang diwarisi dari superclass Circle, sebagai contoh:

```
@Override
public String toString() { // di dalam kelas Cylinder
   return "Cylinder: subclass of " + super.toString() // menggunakan toString() milik
Circle
         + " height=" + height;
}
```

- Cobalah metode toString() tersebut di dalam kelas TestCylinder!

Source Code

```java
/**
 * The Circle class models a circle with a radius and color.
 */
public class Circle { // Save as "Circle.java"
    // private instance variable, not accessible from outside this class
    private double radius;    Field radius can be final
    private String color;

    // Constructors (overloaded)
    /** Constructs a Circle instance with default value for radius and color */
    public Circle() { // 1st (default) constructor
        radius = 1.0;
        color = "red";
    }


    //Getter untuk color(masih belum fix)
    public String getColor(){
        return color;
    }

    //Setter untuk color
    public void setColor(String colorBaru){
        this.color = colorBaru;
    }

    /** Constructs a Circle instance with the given radius and default color */
    public Circle(double r) { // 2nd constructor
        radius = r;
        color = "red";
    }

    /** Returns the radius */
    public double getRadius() {
        return radius;
    }

    /** Returns the area of this Circle instance */
    public double getArea() {
        return radius * radius * Math.PI;
    }

    /** * Return a self-descriptive string of this instance in the form of
     * Circle[radius=?,color=?]
     */
    public String toString() {    Add @Override Annotation
        return "Circle[radius=" + radius + " color=" + color + "]";
    }
}
```

```java
public class Cylinder extends Circle { // Save as "Cylinder.java"
    private double height; // private variable    Field height can be final

    // Constructor with default color, radius and height
    public Cylinder() {
        super(); // call superclass no-arg constructor Circle()
        this.height = 1.0;
    }

    // Constructor with default radius, color but given height
    public Cylinder(double height) {
        super(); // call superclass no-arg constructor Circle()
        this.height = height;
    }

    // Constructor with default color, but given radius, height
    public Cylinder(double radius, double height) {
        super(radius); // call superclass constructor Circle(radius)
        this.height = height;
    }

    // A public method for retrieving the height
    public double getHeight() {
        return height;
    }


     @Override
    public double getArea(){
        return 2 * Math.PI * getRadius() * height + 2 * super.getArea();
    }

    // A public method for computing the volume of cylinder
    // use superclass method getArea() to get the base area
    public double getVolume() {
        return super.getArea() * height;
    }

    @Override
    public String toString() {
    // Memanggil toString() milik Circle (radius & color) + menambahkan height
    return "Cylinder: subclass of " + super.toString() + " height=" + height;
}

}
```

```java
public class TestCylinder { // save as "TestCylinder.java"
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {

        // Declare and allocate a new instance of cylinder
        // with default color, radius, and height
        Cylinder c1 = new Cylinder();
        System.out.println("Cylinder:"
            + " radius=" + c1.getRadius()
            + " height=" + c1.getHeight()
            + " base area=" + c1.getArea()
            + " volume=" + c1.getVolume());

        // Declare and allocate a new instance of cylinder
        // specifying height, with default color and radius
        Cylinder c2 = new Cylinder(height: 10.0);
        System.out.println("Cylinder:"
            + " radius=" + c2.getRadius()
            + " height=" + c2.getHeight()
            + " base area=" + c2.getArea()
            + " volume=" + c2.getVolume());

        // Declare and allocate a new instance of cylinder
        // specifying radius and height, with default color
        Cylinder c3 = new Cylinder(radius: 2.0, height: 10.0);
        System.out.println("Cylinder:"
            + " radius=" + c3.getRadius()
            + " height=" + c3.getHeight()
            + " base area=" + c3.getArea()
            + " volume=" + c3.getVolume());


        Cylinder c4 = new Cylinder(radius: 2.0, height: 5.0);
        // Menguji toString() yang baru
        // Secara otomatis memanggil method toString()
        System.out.println(c4.toString());

        // Menguji apakah getArea() sekarang menghasilkan Luas Permukaan
        System.out.println("Luas Permukaan (Overridden): " + c4.getArea());

        // Menguji apakah getVolume() tetap benar (menggunakan luas alas)
        System.out.println("Volume (Tetap Benar): " + c4.getVolume());
    }
}
```

Screenshot Hasil

```
Cylinder: radius=1.0 height=1.0 base area=12.566370614359172 volume=3.141592653589793
Cylinder: radius=1.0 height=10.0 base area=69.11503837897544 volume=31.41592653589793
Cylinder: radius=2.0 height=10.0 base area=150.79644737231007 volume=125.66370614359172
Cylinder: subclass of Circle[radius=2.0 color=red] height=5.0
Luas Permukaan (Overridden): 87.96459430051421
Volume (Tetap Benar): 62.83185307179586
```

Penjelasan Permasalahan dan Solusi

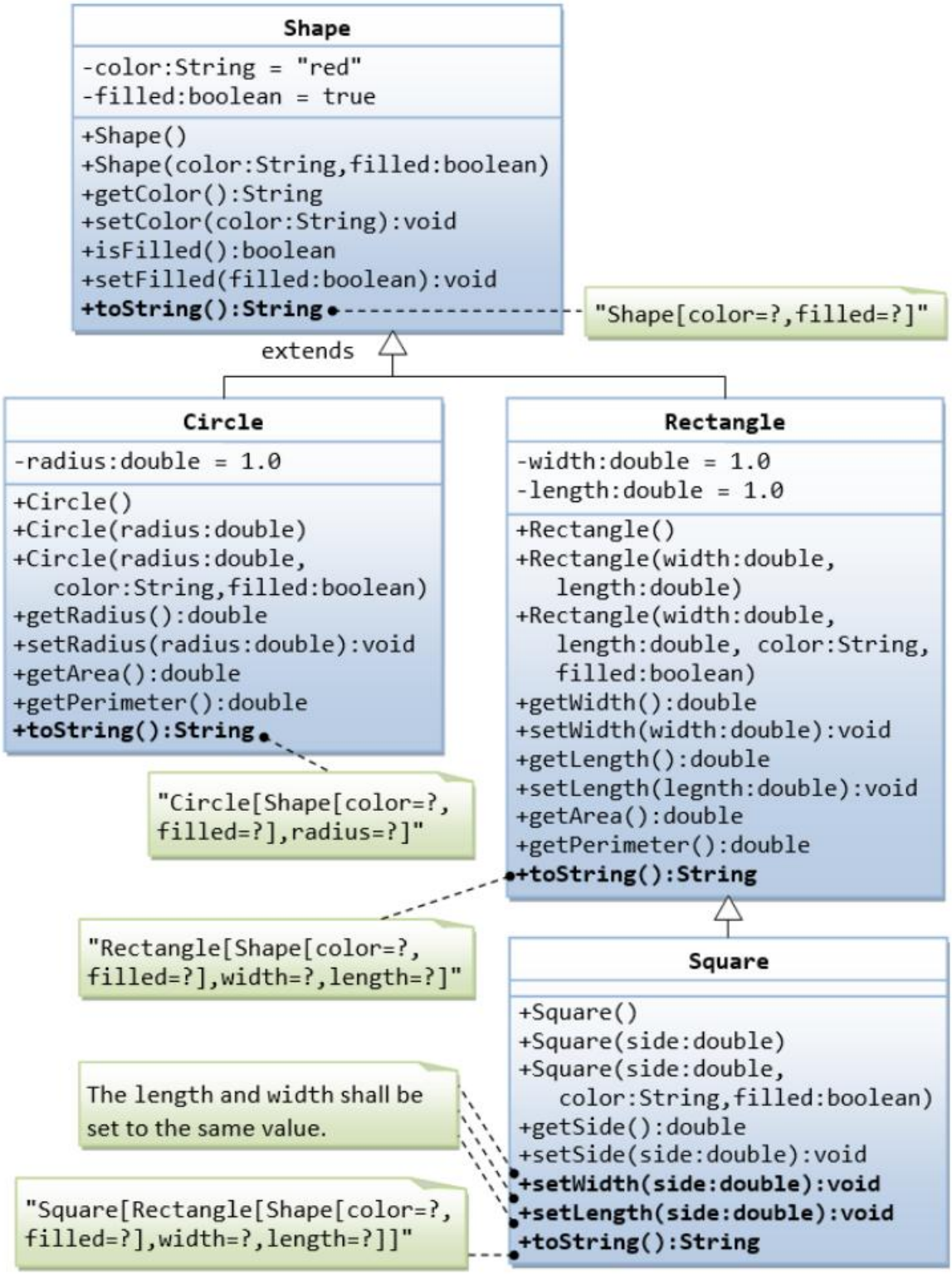
Permasalahan utama dimulai dari keterbatasan struktur data pada class Circle yang belum mampu menyimpan informasi warna dan kurang fleksibel dalam inisialisasi objek, sehingga dilakukan modifikasi dengan menambahkan atribut color yang dilindungi oleh mekanisme Getter dan Setter untuk menjaga prinsip enkapsulasi. Tantangan yang baru muncul saat ingin memodelkan objek Tabung, daripada membangun dari nol, solusi yang efisien adalah menerapkan Inheritance dengan membuat class Cylinder yang mengekstensi Circle. Dalam proses ini, diterapkan konsep Constructor Chaining menggunakan keyword super() untuk memecahkan masalah inisialisasi variabel private milik induk.

Permasalahan logika muncul ketika Cylinder secara otomatis mewarisi metode getArea() yang hanya menghitung luas alas, padahal untuk tabung diperlukan luas permukaan. Solusinya adalah melakukan Method Overriding untuk mendefinisikan ulang rumus luas tersebut. Namun, modifikasi ini menimbulkan efek samping di mana metode getVolume() menjadi tidak akurat karena ikut menggunakan rumus luas

permukaan yang baru. Untuk mengatasinya, digunakan keyword super.getArea() di dalam fungsi volume untuk memanggil kembali logika asli superclass tanpa merusak fungsi luas permukaan pada subclass. Terakhir, untuk memberikan informasi objek yang lengkap, metode toString() juga di-override dengan menggabungkan representasi data dari super.toString() dan atribut tinggi, yang kemudian divalidasi efektivitasnya melalui program pengujian TestCylinder untuk memastikan prinsip Polimorfisme berjalan dengan benar.
Nama Teman Hal yang Dibantu (Opsional)

No. 2 Superclass dan Subclass

Soal Praktikum
Superclass Shape dan Subclass-nya: Circle, Rectangle, dan Square.



A. Instruksi - Hirarki Kelas Shape:

1. Buatlah sebuah *superclass* bernama **Shape** (seperti yang ditunjukkan pada diagram kelas), yang berisi:

   - Dua variabel instans: color (String) dan filled (boolean).
   - Dua *constructor*: sebuah *no-arg (no-argument) constructor* yang menginisialisasi color ke "green" dan filled ke true, dan sebuah *constructor* yang menginisialisasi color dan filled ke nilai yang diberikan.
   - *Getter* dan *setter* untuk semua variabel instans. Berdasarkan konvensi, *getter* untuk variabel boolean xxx disebut isXXX() (bukan getXxx() seperti pada tipe lainnya).
   - Sebuah metode toString() yang mengembalikan "A Shape with color of xxx and filled/Not filled".
   - Tulislah sebuah program uji untuk menguji semua metode yang didefinisikan dalam Shape.
2. Buatlah dua *subclass* dari Shape bernama **Circle** dan **Rectangle**, seperti yang ditunjukkan pada diagram kelas.
   Kelas **Circle** berisi:
   - Sebuah variabel instans radius (double).
   - Tiga *constructor* seperti yang ditunjukkan. *No-arg constructor* menginisialisasi radius ke 1.0.
   - *Getter* dan *setter* untuk variabel instans radius.
   - Metode getArea() dan getPerimeter().
   - *Override* metode toString() yang diwarisi, untuk mengembalikan "A Circle with radius=xxx, which is a subclass of yyy", di mana yyy adalah *output* dari metode toString() milik *superclass*.

   Kelas **Rectangle** berisi:
   - Dua variabel instans width (double) dan length (double).
   - Tiga *constructor* seperti yang ditunjukkan. *No-arg constructor* menginisialisasi width dan length ke 1.0.
   - *Getter* dan *setter* untuk semua variabel instans.
   - Metode getArea() dan getPerimeter().
   - *Override* metode toString() yang diwarisi, untuk mengembalikan "A Rectangle with width=xxx and length=zzz, which is a subclass of yyy", di mana yyy adalah *output* dari metode toString() milik *superclass*.
3. Buatlah sebuah kelas bernama **Square**, sebagai *subclass* dari Rectangle. Yakinkan diri Anda bahwa Square dapat dimodelkan sebagai *subclass* dari Rectangle. Square tidak memiliki variabel instans, tetapi mewarisi variabel instans width dan length dari *superclass*-nya, Rectangle.
   - Sediakan *constructor* yang sesuai (seperti yang ditunjukkan dalam diagram kelas)! Petunjuk:

     ```
     public Square(double side) {
       super(side, side);  // Call superclass Rectangle(double, double)
     ```

}

- *Override* metode toString() untuk mengembalikan "A Square with side=xxx, which is a subclass of yyy", di mana yyy adalah *output* dari metode toString() milik *superclass*!
- Apakah Anda perlu melakukan *override* pada getArea() dan getPerimeter()? Cobalah!
- *Override* setLength() dan setWidth() untuk mengubah width sekaligus length, guna menjaga geometri persegi!

Source Code

```java
public class Shape {
    private String color;
    private  boolean filled;

   public Shape(){
    this.color = "green";
    this.filled = true;
   }

   public Shape(String color, boolean filled){
    this.color = color;
    this.filled = filled;
   }

   public String getColor(){
    return color;
   }

   public void setColor(String color){
    this.color = color;
   }

   public boolean isFilled(){
    return filled;
   }

   public void setFilled(boolean filled){
    this.filled = filled;
   }

   @Override
   public String toString(){
    String statusFilled = filled ? "filled" : "Not filled";
    return "A Shape with color of " + color + " and " + statusFilled;
   }
}


public class Circle extends Shape {
    private double radius;

    public Circle (){
        super();
        this.radius = 1.0;
    }

    public Circle (double radius){
        super();
        this.radius = radius;
    }


    public Circle (double radius, String color, boolean filled){
        super(color, filled);
        this.radius = radius;
    }

    public double getRadius(){
        return radius;
    }

    public void setRadius(double radius){
        this.radius = radius;
    }

    public double getPerimeter(){
        return 2 * Math.PI * radius;
    }

    @Override
    public String toString() {
        // yyy adalah super.toString()
        return "A Circle with radius=" + radius + ", which is a subclass of " + super.toString();
    }
}
```

```java
public class Rectangle extends Shape {
    private double width;
    private double length;

    public Rectangle(){
        super();
        this.width = 1.0;
        this.length = 1.0;
    }

    public Rectangle(double width, double length){
        super();
        this.width = width;
        this.length = length;
    }

    public Rectangle(double width, double length, String color, boolean filled){
        super(color, filled);
        this.width = width;
        this.length = length;
    }

    public double getWidth(){
        return width;
    }

    public void setWidth(double width){
        this.width = width;
    }

    public double getLength(){
        return length;
    }

    public void setLength(double length){
        this.length = length;
    }

    public double getArea() {
        return width * length;
    }

   public double getPerimeter() {
        return 2 * (width + length);
    }

    @Override
    public String toString(){
        return "A Rectangle with width = " + width + " and Length = " + length + ", wich is subclass of " + super.toString();
    }
}
```

```java
public class Square extends Rectangle{

    public Square(double side){
        super(side, side); //call superclass Rectangle(double, double)
    }

    public Square(double side, String color, boolean filled){
        super(side, side, color, filled);
    }

    public double getSide(){
        return getWidth();
    }

    public void setSide(double side){
        super.setWidth(side);
        super.setLength(side);
    }

    @Override
    public void setWidth(double side) {
        setSide(side);
    }

    @Override
    public void setLength(double side) {
        setSide(side);
    }

    // 4. Override toString
    @Override
    public String toString() {
        return "A Square with side=" + getSide() + ", which is a subclass of " + super.toString();
    }
}
```

Screenshot Hasil

```
S1: A Shape with color of green and filled
S2: A Shape with color of red and Not filled
S1 Warna Baru:yellow
S1 apakah Filled? true
S1 setelah di set A Shape with color of yellow and filled
```

Penjelasan Permasalahan dan Solusi

Permasalahan yang dihadapi dalam tugas ini adalah membangun struktur hirarki objek yang efisien dengan menerapkan prinsip dasar pemrograman berorientasi objek yaitu enkapsulasi dan pewarisan serta polimorfisme melalui kelas induk bernama Shape. Kesulitan utama muncul saat mendefinisikan hubungan antar kelas yang memiliki kemiripan namun memiliki spesifikasi berbeda seperti lingkaran dan persegi panjang yang harus mewarisi atribut warna dan status pengisian dari kelas induk namun memiliki rumus perhitungan luas serta keliling yang unik sesuai geometri masing-masing. Masalah teknis lainnya adalah menjaga integritas data pada kelas persegi yang merupakan turunan dari persegi panjang karena sebuah persegi harus selalu memiliki dimensi lebar dan panjang yang sama sehingga perubahan pada salah satu sisi tidak boleh merusak bentuk geometrisnya.

Solusi yang diterapkan adalah menggunakan kata kunci extends untuk menghubungkan kelas anak dengan kelas induk sehingga atribut serta metode umum

tidak perlu ditulis ulang guna meningkatkan efisiensi kode. Untuk mengatasi masalah inisialisasi pada kelas turunan digunakan mekanisme konstruktor berantai melalui kata kunci super yang memungkinkan data dikirimkan kembali ke kelas induk untuk diproses secara benar. Masalah perhitungan luas dan keliling diselesaikan dengan teknik override pada metode terkait di setiap subclass untuk menyediakan logika perhitungan yang akurat bagi lingkaran dan persegi panjang. Khusus untuk kelas persegi solusinya adalah dengan menimpa fungsi pengatur lebar dan panjang agar setiap kali salah satu sisi diubah maka sisi lainnya otomatis mengikuti nilai yang sama melalui metode setSide sehingga karakteristik persegi tetap terjaga. Implementasi metode toString yang memanfaatkan keluaran dari super.toString memastikan bahwa representasi objek di layar memberikan informasi hierarki yang lengkap dan informatif bagi pengguna.
Nama Teman Hal yang Dibantu (Opsional)

No. 3 Multiple Inheritance

Soal Praktikum

Dua contoh di bawah merupakan kelas pertama yang mendeskripsikan sekumpulan Karyawan (Employees) yang bekerja di sebuah pabrik, dan kelas kedua yang mendeskripsikan subset Karyawan yang merupakan Manajer (Manager) di pabrik yang sama. Kedua kelas tersebut mewakili sebuah contoh untuk menunjukkan kekuatan ekspresif dari Pewarisan (Inheritance) dalam Java.
 	
Secara khusus, kelas Manager didefinisikan sebagai subclass dari Employee, dengan tujuan untuk menggunakan kembali perangkat lunak yang telah ditulis untuk kelas Employee.
 	
1. Class Employee.java
class Employee {
  private String name;
  private double salary;
  private int hireday;
  private int hiremonth;
  private int hireyear;
 	
  public Employee(String n, double s, int day, int month, int year) {
  name = n;
  salary = s;
  hireday = day;
  hiremonth = month;
  hireyear = year;
  }
 	
  public void print() {
  System.out.println(name + " " + salary + " " + hireYear());
  }
 	
  public void raiseSalary(double byPercent) {
  salary *= 1 + byPercent / 100;
  }
 	
  public int hireYear() {
  return hireyear;
  }
}
 	
2. Class EmployeeTest.java
public class EmployeeTest {
  public static void main(String[] args) {
  // Mendeklarasikan dan mengalokasikan array untuk 3 objek Employee
  Employee[] staff = new Employee[3];
 	
  // Inisialisasi data karyawan
  staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
  staff[1] = new Employee("Maria Bianchi", 2500000, 1, 12, 1991);
  staff[2] = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);

```
        // Menaikkan gaji setiap staf sebesar 5%
        for (int i = 0; i < 3; i++) {
            staff[i].raiseSalary(5);
        }

        // Mencetak data dari setiap staf
        for (int i = 0; i < 3; i++) {
            staff[i].print();
        }
    }
}
```

3. Class Manager.java

```
import java.util.Calendar;
import java.util.GregorianCalendar;

class Manager extends Employee {
    private String secretaryName;

    public Manager(String n, double s, int d, int m, int y) {
        super(n, s, d, m, y);
        secretaryName = "";
    }

    @Override
    public void raiseSalary(double byPercent) {
        // Menambahkan bonus 1/2% untuk setiap tahun masa kerja
        GregorianCalendar todaysDate = new GregorianCalendar();
        int currentYear = todaysDate.get(Calendar.YEAR);
        double bonus = 0.5 * (currentYear - hireYear());

        super.raiseSalary(byPercent + bonus);
    }

    public String getSecretaryName() {
        return secretaryName;
    }
}
```

4. Class ManagerTest.java

```
public class ManagerTest {
    public static void main(String[] args) {
        // Mendeklarasikan dan mengalokasikan array untuk 3 objek Employee
        Employee[] staff = new Employee[3];

        // Mengisi array dengan kombinasi Employee dan Manager (Polimorfisme)
        staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
        staff[1] = new Manager("Maria Bianchi", 2500000, 1, 12, 1991);
        staff[2] = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);

        // Menaikkan gaji semua staf sebesar 5%
        for (int i = 0; i < 3; i++) {
            staff[i].raiseSalary(5);
        }

        // Mencetak data setiap staf
```

```
        for (int i = 0; i < 3; i++) {
            staff[i].print();
        }
    }
}
```

A. Instruksi 1

Terdapat sebuah abstract class bernama Sortable.

```
abstract class Sortable {
    public abstract int compare(Sortable b);

    public static void shell_sort(Sortable[] a) {
        // Shell sort body
    }
}
```

Ketika Sortable diturunkan ke kelas Employee, metode compare akan diimplementasikan.

```
class Employee extends Sortable {
    /* another methods */

    @Override
    public int compare(Sortable b) {
        Employee eb = (Employee) b;
        if (salary < eb.salary) return -1;
        if (salary > eb.salary) return 1;
        return 0;
    }
}
```

Silakan coba kode di atas! Panggil metode compare, di dalam kelas EmployeeTest!

B. Instruksi 2

Bayangkan kita ingin mengurutkan Manager dengan cara yang serupa:

```
class Managers extends Employee extends Sortable
```

1. Apakah itu akan berhasil?
2. Apa solusi Anda?

Source Code

```
class Employee extends Sortable {
   private String name;
   private double salary;
   private int hireday, hiremonth, hireyear;

   public Employee(String n, double s, int day, int month, int year) {
      name = n;
      salary = s;
      hireday = day;
      hiremonth = month;
      hireyear = year;
```

```
    }

    public String getName() { return name; }
    public double getSalary() { return salary; }

    public void print() {
        System.out.println(name + " " + salary + " " + hireYear());
    }

    public void raiseSalary(double byPercent) {
        salary *= 1 + byPercent / 100;
    }

    public int hireYear() { return hireyear; }

    @Override
    public int compare(Sortable b) {
        Employee eb = (Employee) b;
        if (salary < eb.salary) return -1;
        if (salary > eb.salary) return 1;
        return 0;
    }
}

public class EmployeeTest {
    public static void main(String[] args) {
        Employee[] staff = new Employee[3];
        staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
        staff[1] = new Employee("Maria Bianchi", 2500000, 1, 12, 1991);
        staff[2] = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);

        // Menaikkan gaji
        for (Employee e : staff) e.raiseSalary(5);

        // Menguji metode compare
        int result = staff[0].compare(staff[1]);
        if (result < 0) {
                System.out.println(staff[0].getName() + " gaji lebih kecil dari " +
staff[1].getName());
        }

        // Mencetak data
        for (Employee e : staff) e.print();
    }
}

import java.util.Calendar;
```

```
import java.util.GregorianCalendar;

class Manager extends Employee {
   private String secretaryName = "";

   public Manager(String n, double s, int d, int m, int y) {
      super(n, s, d, m, y);
   }

   @Override
   public void raiseSalary(double byPercent) {
      GregorianCalendar todaysDate = new GregorianCalendar();
      int currentYear = todaysDate.get(Calendar.YEAR);
      double bonus = 0.5 * (currentYear - hireYear());
      super.raiseSalary(byPercent + bonus);
   }

   public String getSecretaryName() { return secretaryName; }
}

public class ManagerTest {
   public static void main(String[] args) {
      Employee[] staff = new Employee[3];

      staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
      staff[1] = new Manager("Maria Bianchi", 2500000, 1, 12, 1991); // Polimorfisme
      staff[2] = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);

      for (int i = 0; i < 3; i++) {
         staff[i].raiseSalary(5);
      }

      for (int i = 0; i < 3; i++) {
         staff[i].print();
      }
   }
}

abstract class Sortable {
   public abstract int compare(Sortable b);

   public static void shell_sort(Sortable[] a) {
      int n = a.length;
      for (int gap = n / 2; gap > 0; gap /= 2) {
         for (int i = gap; i < n; i++) {
            Sortable temp = a[i];
            int j;
```

```
            for (j = i; j >= gap && a[j - gap].compare(temp) > 0; j -= gap) {
              a[j] = a[j - gap];
            }
            a[j] = temp;
          }
        }
      }
    }
```

Screenshot Hasil

```
m\AppData\Roaming\Code\User\workspaceStorage\b0dad04b
Antonio Rossi gaji lebih kecil dari Maria Bianchi
Antonio Rossi 2100000.0 1989
Maria Bianchi 2625000.0 1991
Isabel Vidal 3150000.0 1993
w' '-XX:+ShowCodeDetailsInExcept
Antonio Rossi 2100000.0 1989
Maria Bianchi 3062500.0 1991
Isabel Vidal 3150000.0 1993
```

Penjelasan Permasalahan dan Solusi

Permasalahan utama dalam pengembangan sistem manajemen ini adalah bagaimana menciptakan struktur kode yang dapat digunakan kembali untuk berbagai jenis posisi karyawan sambil tetap memungkinkan adanya perilaku yang spesifik bagi setiap jabatan, seperti perhitungan bonus gaji yang berbeda bagi manajer dibandingkan staf biasa. Tantangan lainnya muncul ketika sistem perlu memiliki kemampuan pengurutan data karyawan berdasarkan besaran gaji, yang membutuhkan standar perbandingan yang konsisten lintas kelas objek agar algoritma pengurutan seperti shell sort dapat bekerja secara efektif. Masalah teknis juga ditemukan pada keterbatasan Java yang tidak mendukung pewarisan berganda, sehingga diperlukan strategi hirarki yang tepat agar sebuah kelas seperti manajer dapat mewarisi sifat karyawan sekaligus memenuhi kontrak kemampuan pengurutan tanpa menyebabkan konflik struktur kode.

Solusi yang diterapkan adalah menggunakan konsep pewarisan atau inheritance dengan menjadikan kelas Employee sebagai induk bagi kelas Manager, sehingga atribut dasar seperti nama dan gaji tidak perlu didefinisikan ulang. Untuk mengatasi perbedaan perilaku perhitungan gaji, dilakukan teknik method overriding pada fungsi raiseSalary di kelas Manager agar dapat menambahkan perhitungan bonus masa kerja secara dinamis menggunakan kalender sistem. Masalah kemampuan pengurutan diselesaikan dengan menciptakan kelas abstrak Sortable sebagai kontrak tertinggi yang mendefinisikan metode compare, di mana kelas Employee kemudian mengimplementasikan metode tersebut untuk membandingkan objek berdasarkan nilai gaji. Dengan struktur ini, prinsip polimorfisme dapat berjalan sehingga array bertipe Employee dapat menampung objek Manager secara fleksibel,

dan fungsi pengurutan tetap dapat berjalan secara transitif karena manajer secara otomatis dianggap sebagai objek yang dapat diurutkan melalui garis keturunan induknya.
Nama Teman Hal yang Dibantu (Opsional)